

DJ AI v2.1

Yapay Zeka Destekli Turkce Muzik Oneri Chatbot'u

Proje Dokumantasyonu & Teknik Analiz

Flask + React | Python | JavaScript | SQLite | Spotify API | OpenAI/Gemini

Icindekiler

1. Proje Ozeti & Amac
2. Sistem Mimarisi
3. Backend - Flask API (app.py)
4. NLP & Sohbet Motoru (analyzer.py)
5. LLM Entegrasyonu (llm_service.py)
6. Veritabani & Hafiza (memory.py)
7. Playlist Sistemi (playlist.py)
8. Demo Veritabani (demo_data.py)
9. Muzik Servisi (music_service.py)
10. Frontend - React Uygulamasi
11. Kimlik Dogrulama Sistemi
12. Veri Akisi - Sohbet Mesaji Yasam Dongusu
13. Sohbet Asama Sistemi (Conversation Stage)
14. Baglam Koruma & Duzeltme Sistemi
15. API Endpoint'leri
16. Kurulum & Calistirma

1. Proje Ozeti & Amac

DJ AI, yapay zeka destekli, Turkce konusan bir muzik oneri chatbot'udur. Kullanicilara dogal dilde sohbet ederek ruh halini, aktivitesini ve muzik tercihlerini anlar, bu bilgilere dayali olarak kisisellestirilmis muzik onnerileri sunar.

Temel Ozellikler

- > Turkce dogal dil anlama - 30+ tur, 12 ruh hali, 17 aktivite, 70+ sanatci anahtar kelime veritabani
- > 3 asamali sohbet akisi - Ruh hali sorgulama > Aktivite sorgulama > Oneri sunma
- > Baglam koruma - Duzeltme, tur degisikligi, reddetme durumlarinda onceki baglami korur
- > Coklu LLM destegi - OpenAI GPT, Google Gemini, yerel Ollama ile sirali yedekleme
- > Spotify entegrasyonu - Sarki zenginlestirme, playlist disa aktarma
- > 278 demo sarki - API anahtari olmadan da calisir
- > Kullanici kimlik dogrulama - Firebase + e-posta/sifre + misafir modu
- > Playlist olusturma - Tercih bazli akilli puanlama sistemi

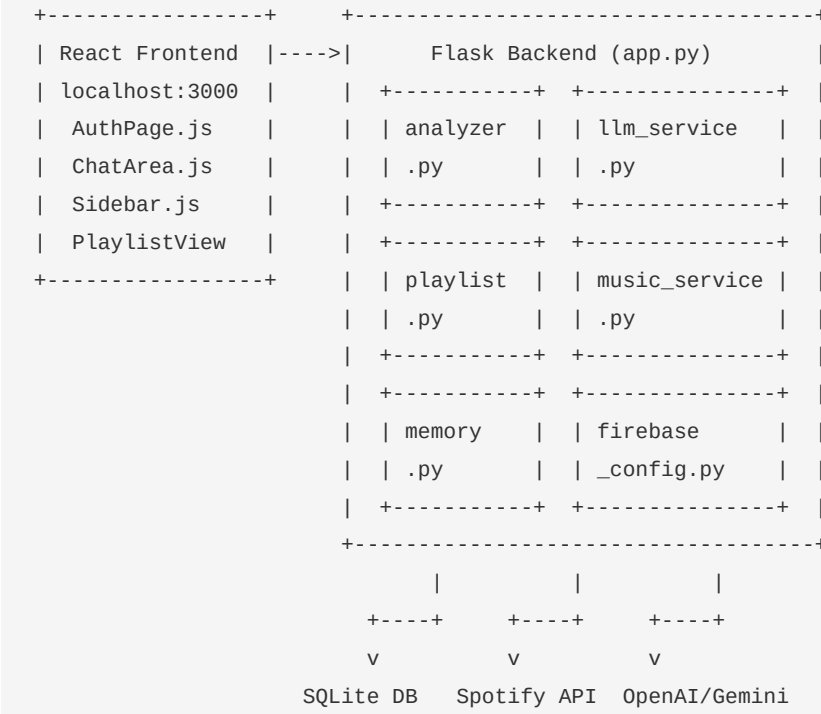
Teknoloji Yigini

Katman	Teknoloji	Amac
Backend	Flask (Python 3.13)	REST API sunucusu
Veritabani	SQLite + Firebase Firestore	Kullanici, sohbet, tercih verileri
AI	OpenAI / Gemini / Ollama	Dogal dil yanitlama
Muzik	Spotify Web API (spotipy)	Canli sarki arama & zenginlestirme
Frontend	React 18 + Tailwind CSS	Kullanici arayuzu
Auth	Firebase Auth + SQLite	Kimlik dogrulama
NL	Ozel NLP (analyzer.py)	Turkce anahtar kelime eslestirme

2. Sistem Mimarisi

DJ AI, üç katmanlı bir mimari kullanır: React frontend, Flask backend API ve veri katmanı. Frontend, backend ile REST API üzerinden haberleşir. Backend, NLP motoru, LLM servisleri, Spotify API'si ve veritabanı arasında köprü görevi görür.

Mimari Diyagram



3. Backend - Flask API (app.py)

app.py, DJ AI'in ana sunucu dosyasidir. Flask ile 15+ REST API endpoint tanımlar, rate limiting uygular, oturum durumu yönetir ve tüm servisleri bir araya getirir.

Sohbet Akisi (chat endpoint)

```
POST /api/chat
1. Kullanici mesajini al
2. Mesaji analiz et (tur, ruh hali, aktivite)
3. Oturum durumunu getir (conversation_stage)
4. LLM yaniti dene (OpenAI > Gemini > Ollama)
5. LLM yoksa ResponseGenerator ile demo yanit uret
6. Oturum durumunu guncelle ve kaydet
7. Yaniti JSON olarak dondur
```

Oturum Durumu Yonetimi

Her sohbet oturumu (session_id + user_id ile anahtarlanmış) bir asama ve baglam objesi tutar. Bu sayede bot, kullanıcının önceki ruh halini ve aktivitesini hatırlar.

```
conversation_states = {
    "user_id:session_id": {
        "stage": "asking_mood|asking_activity|recommending",
        "context": {
            "mood": "uzgun|mutlu|enerjik|...",
            "genre": "rock|pop|rap|...",
            "activity": "ders|spor|gece|...",
            ...
        }
    }
}
```

4. NLP & Sohbet Motoru (analyzer.py)

analyzer.py, DJ AI'nin kalbidir. İki ana sınıf içerir: MessageAnalyzer (anahtar kelime tespiti) ve ResponseGenerator (asama tabanlı yanıt üretimi).

MessageAnalyzer - Anahtar Kelime Tespiti

Kullanıcı mesajını Türkçe karakter normalizasyonu ile işler. Ardından 4 kategoride bulanık eşleştirme (fuzzy matching) yapar:

- > Tur tespiti - 30+ tur (rock, pop, rap, jazz, klasik, lofi, arabesk, anadolu_rock...)
- > Ruh hali tespiti - 12 duygu (mutlu, üzgün, enerjik, romantik, asik, sakin, motivasyon, sinirli, melankolik, nostaljik, guclu, epik)
- > Aktivite tespiti - 17 aktivite (ders, spor, gece, yolculuk, uyku, parti, kahve, yağmur, araba, kosu, oyun, kod, chill, odak, meditasyon...)
- > Sanatçı tespiti - 50+ sanatçı (Queen, Tarkan, Sezen Aksu, Duman, Pink Floyd...)

Türkçe Karakter Normalizasyonu

```

TURKISH_REPLACEMENTS = {
    "c": "ç", "g": "ğ", "i": "ı", "o": "ö",
    "s": "ş", "u": "ü", "I": "İ", "C": "Ç",
    "G": "Ğ", "O": "Ö", "S": "Ş", "U": "Ü"
}

```

Bu sayede kullanıcı Türkçe eklerle yazdığında (örneğin "iyiyim", "mutluyum", "üzgünüm") anahtar kelimeler doğru eşleştirilir.

ResponseGenerator - Asama Tabanlı Yanıt Sistemi

Asama	Acıklama	Gecis Kosulu
asking_mood	Bot ruh halini sorar	Ruh hali algılandığında > asking_activity
asking_activity	Bot ne yaptığını sorar	Aktivite algılandığında > recommending
recommending	Müzik önerisi sunar	Düzeltilme/tur değişikliğinde kalır

Düzeltilme & Bağlam Koruma

- > Tur değişikliği - "rap olsun", "rock değil" > yeni tur, mood/activity korunur
- > Reddetme - "olmadı", "istemem" > alternatif öneri sunar
- > Devam - "tamam", "evet" > sohbeti sürdürür
- > Daha fazla - "daha", "baska" > aynı bağlamda yeni öneri sunar

5. LLM Entegrasyonu (llm_service.py)

LLMClient, üç farklı AI sağlayıcısını sıralı yedekleme (fallback) zinciri ile kullanır. Bir sağlayıcı başarısız olursa veya yanıt vermezse, bir sonraki sağlayıcı denir.

Yedekleme Zinciri

```
OpenAI (GPT-3.5/4) > Gemini (2.0-flash) > Ollama (llama3) > Demo Modu
      başarısız      başarısız      başarısız
```

Sistem Prompt'u

LLM'ler, Türkçe konuşan sıcak ve doğal bir müzik asistanı olarak yönlendirilir. Ana kurallar: Robot gibi konuşma, şarkı ismi uydurma, önce anlamaya çalış.

Streaming Destegi

OpenAI için streaming modu mevcuttur (/api/chat/stream endpoint'i). Yanıtlar token token gönderilir, kullanıcı uzun yanıtları beklemeden görebilir.

6. Veritabani & Hafiza (memory.py)

MemoryStore sinifi, çift katmanlı veri saklama sağlar: SQLite (varsayılan) ve Firebase Firestore (isteğe bağlı).

Veritabani Tabloları

Tablo	Amac	Anahtar Alanlar
users	Kullanıcı bilgileri	user_id (PK), display_name
accounts	E-posta/sifre girişi	user_id (PK), email (UQ), password_hash
preferences	Kullanıcı tercihleri	user_id, category, value
conversations	Sohbet geçmişi	user_id, session_id, role, content, analysis
playlists	Oluşturulan playlistler	id, user_id, name, preferences
playlist_songs	Playlist şarkıları	playlist_id, song_name, artist, genre

7. Playlist Sistemi (playlist.py)

PlaylistGenerator, kullanıcı tercihlerine göre demo veritabanından şarkıları puanlayarak kişiselleştirilmiş playlistler oluşturur.

Kriter	Puan	Acıklama
Tur eslesmesi	+3/tur	Secilen türlere uygun şarkılar
Ruh hali eslesmesi	+2	Secilen ruh haline uygun şarkılar
Aktivite eslesmesi	+2	Secilen aktiviteye uygun şarkılar
Sanatçı eslesmesi	+5	Tam sanatçı eslesmesi
Dil eslesmesi	+1	Türkçe/İngilizce tercihi eslesmesi
Sevilmeyen tür	-5	Kullanıcının sevmediği türlerde cezalandırma

8. Demo Veritabanı (demo_data.py)

API anahtarı olmadan da uygulamanın çalışabilmesi için 278 demo şarkı tanımlanmıştır. Her şarkı şu metadata'ya sahiptir: name, artist, album, genres[], moods[], activities[], language, cover_url, spotify_url, duration, reason.

- > 278 şarkı - Türkçe ve İngilizce
- > 30+ tür kategorisi (rock, pop, rap, jazz, klasik, lofi, arabesk, anadolu_rock...)
- > 12 ruh hali etiketi
- > 17 aktivite etiketi
- > Spotify URL'leri (mevcut olan şarkılar için)

9. Muzik Servisi (music_service.py)

MusicClient, iki katmanlı bir mimari sunar: SpotifyService (canlı API) ve DemoMusicService (yedek). Spotify erişimi varsa canlı arama ve öneriler yapılır, yoksa demo veritabanı kullanılır.

- > spotipy kutuphanesi ile Spotify Web API'ye erişim
- > Şarkı arama (search_tracks) - sanatçı veya genel arama
- > Öneri alma (get_recommendations) - tür bazlı öneriler
- > Şarkı zenginleştirme (enrich_songs) - demo şarkılarına kapak resmi ve Spotify link ekleme
- > Otomatik token yenileme - süresi dolan access_token'ları refresh_token ile yeniler

10. Frontend - React Uygulaması

DJ AI'nin frontend'i React 18 ve Tailwind CSS ile oluşturulmuştur.

Bileşen Hiyerarşisi

```
index.js
+-- AuthProvider (context)
+-- ThemeProvider (context)
  +-- App.js (ana durum yönetimi)
    |-- [giris yapilmedi] > AuthPage
    |   +-- Giriş formu (e-posta/sifre + Google SSO)
    |   +-- Kayıt formu
    |   +-- Ücretsiz Dene (misafir modu)
    |
    +-- [giris yapildi] > Ana duzen
        +-- Sidebar (kategoriler, playlistler, sanatçılar)
        +-- ChatArea (mesaj listesi + girdi)
        +-- PlaylistView (oluşturulan playlist)
        +-- SharePlaylistModal (kaydetme/paylaşma)
```

Özellikler

- > Karanlık/aydınlık tema desteği (localStorage ile kalıcı)
- > Yanıt yazıyor animasyonu (TypingIndicator)
- > Sarki mini kartları (kapak resmi, sanatçı, tür etiketi)
- > Kategori butonları ile hızlı mesaj gönderme
- > Responsive tasarım (mobil uyumlu)

11. Kimlik Doğrulama Sistemi

Üç farklı giriş yöntemi desteklenir. Firebase yapılandırılmamışsa sistem SQLite tabanlı e-posta/sifre ile çalışır.

Yöntem	Mekanizma	Veri Saklama
Google SSO	Firebase Auth popup	Firebase + SQLite senkron
E-posta/Sifre	Backend hash doğrulama	SQLite accounts tablosu
Misafir	UUID oluşturma	localStorage (sunucu kaydı yok)

Sifre Güvenliği

- > PBKDF2-HMAC-SHA256 ile tuzlananmış (salted) hash - 100.000 iterasyon
- > Eski SHA256 hash desteği (geriye dönük uyumluluk)
- > Firebase ID token doğrulama (Google SSO için)

12. Veri Akisi - Sohbet Mesaji Yasam Dongusu

Bir kullanıcının mesaj göndermesinden yaniti gormesine kadar olan adimler:

- > Adim 1: Kullanici ChatArea'ya mesaj yazar ve gonderir
- > Adim 2: Frontend POST /api/chat istegi yapar (message, session_id, user_id)
- > Adim 3: Backend mesaji memory'ye kaydeder (save_message)
- > Adim 4: MessageAnalyzer ile mesaji analiz eder (tur, ruh hali, aktivite tespiti)
- > Adim 5: Oturum durumunu getirir (conversation_states dict'inden)
- > Adim 6: Playlist istegi mi kontrol eder (is_playlist_request)
- > Adim 7: Playlist istegi varsa > PlaylistGenerator ile playlist olusturur
- > Adim 8: Playlist istegi yoksa > ResponseGenerator ile yanit uretir (asama tabanli)
- > Adim 9: LLM modu aktifse > once LLM yaniti denenir (OpenAI > Gemini > Ollama)
- > Adim 10: LLM basarisizsa > Spotify API denenir (sarki arama/oneri)
- > Adim 11: Spotify da yoksa > Demo veritabanindan sarki secilir
- > Adim 12: Yanit ve sarkilar JSON olarak frontend'e dondurulur
- > Adim 13: Frontend mesaji chat listesine ekler, sarki kartlarini gosterir

13. Sohbet Asama Sistemi (Conversation Stage)

Bot, her mesajda konuşmanın hangi aşamasında olduğunu takip eder. Bu sayede aceleci yanıt vermek yerine önce kullanıcıyı anlar.

Asama Diyagramı

```

+-----+
Kullanıcı ---->| asking_mood |<-- Yeni sohbet başlangıcı
girisi      | (Ruh hali sor) |
            +-----+-----+
                | Ruh hali algılandı
                v
            +-----+-----+
                | asking_activity |
                | (Aktivite sor)  |
            +-----+-----+
                | Aktivite algılandı
                v
            +-----+-----+
Düzeltilme ---->| recommending |<-- Tür değişikliğinde kalır
Tür değişikliği| (Öneri sun)   |
Reddetme      +-----+-----+

```

Örnek Sohbet Akışı

Kullanıcı: Merhaba > Bot: Selamlar! Ruh halin nasıl bugün? (asking_mood)

Kullanıcı: Uzunum > Bot: Anladım... Peki ne yapıyorsun? (asking_activity)

Kullanıcı: Ders çalışıyorum > Bot: Öneriler: 1. Breathe 2. Yesterday 3. Comfortably Numb (recommending)

Kullanıcı: Bunlar rock değil, rap olsun > Bot: Haklısın, düzeltiyorum (recommending, bağlam korunur)

14. Bağlam Koruma & Düzeltme Sistemi

Bot, öneri sonrası kullanıcının düzeltme, reddetme veya tür değişikliğe yapmasından önceki bağlamın başına sarmas.

Düzeltilme Algoritması

Kullanıcı Derse	Algılama	Bot Tepkisi
"Bunlar rock değil"	Tür düzeltme	Yeni türde öneri, mood/activity korunur
"Rap olsun"	Tür değişikliği	Yeni türde öneri
"Olmadı, beğenmedim"	Reddetme	Alternatif öneri
"Daha fazla öneri"	Devam	Aynı bağlamda yeni öneri
"Tamam, evet"	Kabul	Sohbeti sürdürür
"Türkçe olsun"	Dil değişikliği	Türkçe filtre ile öneri

15. API Endpoint'leri

DJ AI backend'i 20+ REST API endpoint sunar:

Sohbet & Oneri

Endpoint	Method	Aciklama
/api/chat	POST	Ana sohbet endpoint'i
/api/chat/stream	POST	Streaming sohbet (SSE)
/api/categories	GET	Kategori listesi
/api/artists	GET	Sanatci listesi
/api/status	GET	Sistem durumu

Kimlik Dogrulama

/api/auth/register	POST	E-posta/sifre ile kayit
/api/auth/login	POST	E-posta/sifre ile giris
/api/auth/verify	POST	Firebase token dogrulama
/api/auth/status	GET	Firebase yapilandirma durumu
/api/auth/spotify-link	POST	Spotify hesabi baglama
/api/auth/spotify-unlink	POST	Spotify baglantisini kaldırma

Kullanici & Playlist

/api/user/:id/preferences	GET	Kullanici tercihleri
/api/user/:id/playlists	GET	Kullanici playlistleri
/api/playlist/:id	GET	Tekil playlist detayi
/api/playlist/:id/rename	POST	Playlist isim degisikligi
/api/playlist/:id/delete	POST	Playlist silme
/api/playlist/suggest-name	POST	AI playlist isim önerisi
/api/reset/:session	POST	Oturumu sifirlama
/api/spotify/export-playlist	POST	Spotify'a playlist disa aktarma

16. Kurulum & Çalıştırma

Gereksinimler

- > Python 3.10+
- > Node.js 16+
- > Spotify API anahtarı (isteğe bağlı)
- > OpenAI veya Gemini API anahtarı (isteğe bağlı)

Backend Kurulumu

```
cd backend
pip install -r requirements.txt
cp .env.example .env          # API anahtarlarını güncelle
python app.py                 # Geliştirme modu
python run_production.py      # Üretim modu (Waitress)
```

Frontend Kurulumu

```
cd frontend
npm install
npm start                     # Geliştirme modu (localhost:3000)
npm run build                  # Üretim derlemesi
```

Demo Modu

API anahtarları olmadan uygulama demo modunda çalışır. Bu modda: LLM yanıtları kullanılmaz, ResponseGenerator demo yanıtlar üretir; Spotify API çağrıları yapılmaz, 278 demo şarkı kullanılır; Firebase kimlik doğrulama kullanılmaz, e-posta/sifre ve misafir modu çalışır.